

Dicas Práticas sobre Qualidade de Código

Inspiradas em factos reais

Laboratórios de Informática I

28 de Novembro de 2018

MIEI - UM

Exemplo 1

“Eu uso sempre guardas”

```
instrucao :: Instrucao -> Editor -> Editor
instrucao Roda (Editor a b c d e) | b==C = Editor a D c d e
                                   | b==D = Editor a B c d e
                                   | b==B = Editor a E c d e
                                   | b==E = Editor a C c d e
```

Melhoria 1: utilizar padrões

```
instrucao :: Instrucao -> Editor -> Editor
instrucao Roda (Editor a C c d e) = Editor a D c d e
instrucao Roda (Editor a D c d e) = Editor a D c d e
instrucao Roda (Editor a B c d e) = Editor a D c d e
instrucao Roda (Editor a E c d e) = Editor a D c d e
```

Melhoria 2: introduzir case

```
instrucao :: Instrucao -> Editor -> Editor
instrucao Roda (Editor a b c d e) = case b of
    C -> Editor a D c d e
    D -> Editor a B c d e
    B -> Editor a E c d e
    E -> Editor a C c d e
```

Melhoria 3: introduzir uma função auxiliar

```
instrucao :: Instrucao -> Editor -> Editor
instrucao Roda (Editor a b c d e) = Editor a (rodaDirecao b) c d e

rodaDirecao :: Direcao -> Direcao
rodaDirecao C = D
rodaDirecao D = B
rodaDirecao B = E
rodaDirecao E = C
```

Melhoria 4: utilizar notação de records

```
instrucao :: Instrucao -> Editor -> Editor
instrucao Roda ed = ed { direcaoEditor = rodaDirecao (direcaoEditor ed) }

rodaDirecao :: Direcao -> Direcao
rodaDirecao C = D
rodaDirecao D = B
rodaDirecao B = E
rodaDirecao E = C
```

Exemplo 2

“Eu misturo casos independentes”

```
moveTanque :: Direcao -> Jogador -> Jogador
moveTanque d (Jogador (x,y) dir v l c)
  | d/=dir      = (Jogador (x,y) d v l c)
  | d==dir && d==C = (Jogador (x-1,y) dir v l c)
  | d==dir && d==B = (Jogador (x+1,y) dir v l c)
  | d==dir && d==E = (Jogador (x,y-1) dir v l c)
  | d==dir && d==D = (Jogador (x,y+1) dir v l c)
```

Melhoria 1: agrupar casos

```
moveTanque :: Direcao -> Jogador -> Jogador
moveTanque d (Jogador (x,y) dir v l c)
  | d/=dir      = Jogador (x,y) d v l c -- roda
  | otherwise = case d of                -- move
    C -> Jogador (x-1,y) dir v l c
    B -> Jogador (x+1,y) dir v l c
    E -> Jogador (x,y-1) dir v l c
    D -> Jogador (x,y+1) dir v l c
```

Melhoria 2: utilizar notação de records

```
moveTanque :: Direcao -> Jogador -> Jogador
moveTanque d jo
  | d/=dir      = jo { direcaoJogador = d } -- roda
  | otherwise = case d of                  -- move
    C -> jo { posicaoJogador = (x-1,y) }
    B -> jo { posicaoJogador = (x+1,y) }
    E -> jo { posicaoJogador = (x,y-1) }
    D -> jo { posicaoJogador = (x,y+1) }
where
  dir = direcaoJogador jo
  (x,y) = posicaoJogador jo
```

Melhoria 3: introduzir função auxiliar

```
moveTanque :: Direcao -> Jogador -> Jogador
moveTanque d jo
  | d/=dir    = jo { direcaoJogador = d }           -- roda
  | otherwise = jo { posicaoJogador = movePosicao pos } -- move
where
  dir = direcaoJogador jo
  pos = posicaoJogador jo

movePosicao Posicao -> Direcao -> Posicao
movePosicao (x,y) d = case d of
  C -> (x-1,y)
  B -> (x+1,y)
  E -> (x,y-1)
  D -> (x,y+1)
```

Melhoria 4: reutilizar funções previamente definidas

```
moveTanque :: Direcao -> Jogador -> Jogador
moveTanque d jo
  | d/=dir    = jo { direcaoJogador = d }           -- roda
  | otherwise = jo { posicaoJogador = movePosicao pos } -- move
where
  dir = direcaoJogador jo
  pos = posicaoJogador jo

movePosicao :: Posicao -> Direcao -> Posicao
movePosicao p d = p 'somaVetores' (direcaoParaVetor d)
```

Exemplo 3

“Eu sou demasiado específico”

```
temChoque :: [Disparo] -> Bool
temChoque [] = False
temChoque (DisparoChoque j t:s) = True
temChoque (DisparoCanhao j t d:s) = temChoque s
temChoque (DisparoLaser j t d:s) = temChoque s
```

Melhoria 1: abstrair variáveis e agrupar casos similares

```
temChoque :: [Disparo] -> Bool
temChoque [] = False
temChoque (DisparoChoque _ _):_ = True
temChoque (_:s) = temChoque s
```

Melhoria 2: reutilizar funções predefinidas

```
temChoque :: [Disparo] -> Bool
temChoque = any eChoque
  where
    eChoque :: Disparo -> Bool
    eChoque (DisparoChoque _ _) = True
    eChoque _                    = False
```

Exemplo 4

“Eu uso sempre if-then-else para funções booleanas”

```
eBordaMatriz :: Posicao -> Matriz a -> Bool
eBordaMatriz (m,n) a =
    if (ePosicaoMatrizValida (m,n) a) == True
    then m == 0 || m == (fst(dimensaoMatriz a)-1)
        || n == 0 || n == (snd(dimensaoMatriz a)-1)
    else False
```

Melhoria 1: eliminar testes redundantes

```
eBordaMatriz :: Posicao -> Matriz a -> Bool
eBordaMatriz (m,n) a =
    if (ePosicaoMatrizValida (m,n) a)
    then m == 0 || m == (fst(dimensaoMatriz a)-1)
        || n == 0 || n == (snd(dimensaoMatriz a)-1)
    else False
```

Melhoria 2: eliminar condicional

```
eBordaMatriz :: Posicao -> Matriz a -> Bool
eBordaMatriz (m,n) a = ePosicaoMatrizValida (m,n) a
                        && ( m == 0 || m == fst (dimensaoMatriz a) - 1
                          || n == 0 || n == snd (dimensaoMatriz a) - 1
                          )
```

Melhoria 3: utilizar where

```
eBordaMatriz :: Posicao -> Matriz a -> Bool
eBordaMatriz (m,n) a = ePosicaoMatrizValida (m,n) a
                        || m == 0
                        || m == l - 1
                        || n == 0
                        || n == c - 1

where
  (l,c) = dimensaoMatriz a
```

Exemplo 5

“Eu não sou sucinto”

```
temMunicoesChoque :: Jogador -> Bool
temMunicoesChoque (Jogador (x,y) direcao vida laser choque) =
    if (choque > 0) then True else False
```

Melhoria 1: eliminar condicional e variáveis não utilizadas

```
temMunicoesChoque :: Jogador -> Bool
temMunicoesChoque (Jogador _ _ _ _ choque) = choque > 0
```

Melhoria 2: utilizar projeções de records

```
temMunicoesChoque :: Jogador -> Bool
temMunicoesChoque jo = choquesJogador jo > 0
```

Exemplo 6

“Eu enumero todos os elementos de uma sequência”

```
posicoesChoque :: Jogador -> [Posicao]
posicoesChoque (Jogador (x,y) k z t u) =
    [(x-3,y-3),(x-3,y-2),(x-3,y-1),(x-3,y),(x-3,y+1),(x-3,y+2),(x-3,y+3)
    ,(x-2,y-3),(x-2,y-2),(x-2,y-1),(x-2,y),(x-2,y+1),(x-2,y+2),(x-2,y+3)
    ,(x-1,y-3),(x-1,y-2),(x-1,y-1),(x-1,y),(x-1,y+1),(x-1,y+2),(x-1,y+3)
    ,(x,y-3),(x,y-2),(x,y-1),(x,y),(x,y+1),(x,y+2),(x,y+3)
    ,(x+1,y-3),(x+1,y-2),(x+1,y-1),(x+1,y),(x+1,y+1),(x+1,y+2),(x+1,y+3)
    ,(x+2,y-3),(x+2,y-2),(x+2,y-1),(x+2,y),(x+2,y+1),(x+2,y+2),(x+2,y+3)
    ,(x+3,y-3),(x+3,y-2),(x+3,y-1),(x+3,y),(x+3,y+1),(x+3,y+2),(x+3,y+3)]
```

Melhoria 1: calcular a sequência (listas por compreensão)

```
posicoesChoque :: Jogador -> [PosicaoGrelha]
posicoesChoque jo = [ (x',y') | x' <- [x-3..x+3], y' <- [y-3..y+3] ]
  where
    (x,y) = posicaoJogador jo
```

**Os maus hábitos saem caro e
descontrolam-se rapidamente...**

```

intmain() {
    int n;
    while (n = read(), n) {
        int a, b, c, d;
        while (a = read(), a) {
            while (b = read(), b) {
                while (c = read(), c) {
                    while (d = read(), d) {
                        // ... (repeated code for each case) ...
                    }
                }
            }
        }
    }
}

```

The code is a nested loop structure that reads four integers (a, b, c, d) and then processes them. The processing part is repeated for each combination of (a, b, c, d). The code is highly repetitive, with many lines of code being identical or very similar. The code is written in C++ and uses standard library functions like `read()` and `write()`.

9 casos, >500 colunas!

```
jogada :: Int -> Jogada -> Estado -> Estado
(jogada) p (Movimenta D) (Estado m j dis) | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) == D &&
  => (isnextposVazia (encontraPosicaoMatriz (a+1,b+1) m) (encontraPosicaoMatriz (a,b+1) m)) && not ((isAnyTrue (map
  => (isposicaoocupadadetanks [(x,y+2),(x+1,y+2),(x-1,y+2)]) (tirar (encontraJogador p j) j)))) && not (isposicaoatualcomchoques
  => (posicaoJogador (encontraJogador p j)) j (disparosChoque dis)) = (Estado m (atualizaJogador p (atualizaPosicaoJogador D
  => (encontraJogador p j)) j) dis)
  | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) /= D = (Estado m
  => (atualizaJogador p (atualizadirecaoJogador D (encontraJogador p j)) j) dis)
  | otherwise = (Estado m j dis)
  where (a,b) = posicaoGrelhaMapa (posicaoJogador (encontraJogador p j))
        (x,y) = posicaoJogador (encontraJogador p j)
(jogada) p (Movimenta E) (Estado m j dis) | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) == E &&
  => (isnextposVazia (encontraPosicaoMatriz (a+1,b-2) m) (encontraPosicaoMatriz (a,b-2) m)) && not (isAnyTrue (map (isposicaoocupadadetanks
  => [(x,y-2),(x+1,y-2),(x-1,y-2)]) (tirar (encontraJogador p j) j)))) && not (isposicaoatualcomchoques (posicaoJogador (encontraJogador p
  => j)) j (disparosChoque dis)) = (Estado m (atualizaJogador p (atualizaPosicaoJogador E (encontraJogador p j)) j) dis)
  | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) /= E = (Estado m
  => (atualizaJogador p (atualizadirecaoJogador E (encontraJogador p j)) j) dis)
  | otherwise = (Estado m j dis)
  where (a,b) = posicaoGrelhaMapa (posicaoJogador (encontraJogador p j))
        (x,y) = posicaoJogador (encontraJogador p j)
(jogada) p (Movimenta C) (Estado m j dis) | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) == C &&
  => (isnextposVazia (encontraPosicaoMatriz (a-2,b-1) m) (encontraPosicaoMatriz (a-1,b) m)) && not (isAnyTrue (map (isposicaoocupadadetanks
  => [(x-2,y),(x-2,y+1),(x-2,y-1)]) (tirar (encontraJogador p j) j)))) && not (isposicaoatualcomchoques (posicaoJogador (encontraJogador p
  => j)) j (disparosChoque dis)) = (Estado m (atualizaJogador p (atualizaPosicaoJogador C (encontraJogador p j)) j) dis)
  | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) /= C = (Estado m
  => (atualizaJogador p (atualizadirecaoJogador C (encontraJogador p j)) j) dis)
  | otherwise = (Estado m j dis)
  where (a,b) = posicaoGrelhaMapa (posicaoJogador (encontraJogador p j))
        (x,y) = posicaoJogador (encontraJogador p j)
(jogada) p (Movimenta B) (Estado m j dis) | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) == B &&
  => (isnextposVazia (encontraPosicaoMatriz (a+1,b+1) m) (encontraPosicaoMatriz (a+1,b) m)) && not (isAnyTrue (map (isposicaoocupadadetanks
  => [(x+2,y),(x+2,y+1),(x+2,y-1)]) (tirar (encontraJogador p j) j)))) && not (isposicaoatualcomchoques (posicaoJogador (encontraJogador p
  => j)) j (disparosChoque dis)) = (Estado m (atualizaJogador p (atualizaPosicaoJogador B (encontraJogador p j)) j) dis)
  | vidasJogador (encontraJogador p j) > 0 && direcaoJogador (encontraJogador p j) /= B = (Estado m
  => (atualizaJogador p (atualizadirecaoJogador B (encontraJogador p j)) j) dis)
  | otherwise = (Estado m j dis)
  where (a,b) = posicaoGrelhaMapa (posicaoJogador (encontraJogador p j))
        (x,y) = posicaoJogador (encontraJogador p j)
(jogada) p (Dispara Choque) (Estado m j dis) = if jogadorTemChoque (encontraJogador p j) then (Estado m (atualizaJogador p (jogadorgastouChoque
  => (encontraJogador p j)) j) (dis ++ [DisparoChoque p 5])) else (Estado m j dis)
(jogada) p (Dispara Canhao) (Estado m j dis) = (Estado m j (dis ++ [DisparoCanhao p (posicaoTiro (encontraJogador p j)) (direcaoJogador
  => (encontraJogador p j))]))
(jogada) p (Dispara Laser) (Estado m j dis) = if jogadorTemLaser (encontraJogador p j) then (Estado m (atualizaJogador p (jogadorgastouLaser
  => (encontraJogador p j)) j) (dis ++ [DisparoLaser p (posicaoDisparo (posicaoJogador (encontraJogador p j)) (direcaoJogador
  => (encontraJogador p j))]) (direcaoJogador (encontraJogador p j)))])) else (Estado m j dis)
```

>600 colunas!

```
descomprime :: String -> Estado
descomprime j | ((stringsemdisparoscanhao j) && (stringsemdisparoslaser j)) = Estado (descomprimemapa j) (map (descomprimeJogador (tail
    ↳ (dropWhile isnecessario j))) (listimportante 0 (length (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5
    ↳ j))))::[(Int,Int)])))) (map (descomprimedisparosChoque (tail (dropWhile isnecessario4 j))) (listimportante 0 (comprimentotalista3
    ↳ (tail(dropWhile isnecessario4 j))))
    ↳ | ((stringsemdisparoscanhao j) && (stringsemdisparoschoque j)) = Estado (descomprimemapa j) (map (descomprimeJogador (tail
    ↳ (dropWhile isnecessario j))) (listimportante 0 (length (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5
    ↳ j))))::[(Int,Int)])))) (map (descomprimedisparosLaser (tail (dropWhile isnecessario3 j))) (listimportante 0 (comprimentotalista2
    ↳ (tail (dropWhile isnecessario3 j))))
    ↳ | ((stringsemdisparoslaser j) && (stringsemdisparoschoque j)) = Estado (descomprimemapa j) (map (descomprimeJogador (tail
    ↳ (dropWhile isnecessario j))) (listimportante 0 (length (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5
    ↳ j))))::[(Int,Int)])))) (map (descomprimedisparosCanhao (tail(dropWhile isnecessario2 j))) (listimportante 0 (comprimentotalista
    ↳ (tail (dropWhile isnecessario2 j))))
    ↳ | stringsemdisparoscanhao j = Estado (descomprimemapa j) (map (descomprimeJogador (tail (dropWhile isnecessario j)))
    ↳ (listimportante 0 (length (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5 j))))::[(Int,Int)])))) (map
    ↳ (descomprimedisparosLaser (tail (dropWhile isnecessario3 j))) (listimportante 0 (comprimentotalista2 (tail (dropWhile isnecessario3
    ↳ j)))) ++ (map (descomprimedisparosChoque (tail (dropWhile isnecessario4 j))) (listimportante 0 (comprimentotalista3 (tail(dropWhile
    ↳ isnecessario4 j)))))
    ↳ | stringsemdisparoschoque j = Estado (descomprimemapa j) (map (descomprimeJogador (tail (dropWhile isnecessario j)))
    ↳ (listimportante 0 (length (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5 j))))::[(Int,Int)])))) (map
    ↳ (descomprimedisparosCanhao (tail(dropWhile isnecessario2 j))) (listimportante 0 (comprimentotalista (tail (dropWhile isnecessario2
    ↳ j)))) ++ (map (descomprimedisparosLaser (tail (dropWhile isnecessario3 j))) (listimportante 0 (comprimentotalista2 (tail (dropWhile
    ↳ isnecessario3 j)))))
    ↳ | stringsemdisparoslaser j = Estado (descomprimemapa j) (map (descomprimeJogador (tail (dropWhile isnecessario j)))
    ↳ (listimportante 0 (length (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5 j))))::[(Int,Int)])))) (map
    ↳ (descomprimedisparosCanhao (tail(dropWhile isnecessario2 j))) (listimportante 0 (comprimentotalista (tail (dropWhile isnecessario2
    ↳ j)))) ++ (map (descomprimedisparosChoque (tail (dropWhile isnecessario4 j))) (listimportante 0 (comprimentotalista3 (tail(dropWhile
    ↳ isnecessario4 j)))))
    ↳ | stringsemdisparos j = Estado (descomprimemapa j) (map (descomprimeJogador (tail (dropWhile isnecessario j))) (listimportante
    ↳ 0 (length (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5 j))))::[(Int,Int)])))) []
    ↳ | otherwise = Estado (descomprimemapa j) (map (descomprimeJogador (tail (dropWhile isnecessario j))) (listimportante 0 (length
    ↳ (read ( '[' : (takeWhile isnecessario5 (tail ( dropWhile isnecessario5 j))))::[(Int,Int)])))) (map (descomprimedisparosCanhao
    ↳ (tail(dropWhile isnecessario2 j))) (listimportante 0 (comprimentotalista (tail (dropWhile isnecessario2 j)))) ++ (map
    ↳ (descomprimedisparosLaser (tail (dropWhile isnecessario3 j))) (listimportante 0 (comprimentotalista2 (tail (dropWhile isnecessario3
    ↳ j)))) ++ (map (descomprimedisparosChoque (tail (dropWhile isnecessario4 j))) (listimportante 0 (comprimentotalista3 (tail(dropWhile
    ↳ isnecessario4 j)))))
```

Não se esqueçam de:

- utilizar padrões
- utilizar guardas e condicionais com moderação
- separar o código em diversas funções
- conhecer e reutilizar funções pré-definidas
- importar e reutilizar tarefas anteriores
- simplificar código à medida que aprendem novos conceitos

No fundo:

- pensem antes de programar
- não sejam fãs de *copy-paste*
- não reinventem a roda (pelo menos não mais do que uma vez!)
- sejam preguiçosos (desde que não copiem!)